JMORF/MORS — Morpho-Syntax

The Structure of the Lexicon II
Lexical Rules

Francis Bond
Palacký University
https://fcbond.github.io/
bond@ieee.org

Lecture 8
Location: KB-1.11

Overview

1. **Default Inheritance**: how ' / ' constraints combine

2. **Lexical Rules**: the Plural Noun LR with everything it inherits, licensing ***cats***, constant lexemes, ARG-ST and the ARP, the feature FORM
   Activity: the word structure for ***writers***

3. **Appendix**: the other lexical rules so far, for reference

Then: the passive (a lexical rule at work).

Default Inheritance

Default Inheritance

Why do we have default inheritance?

Default Inheritance

Why do we have default inheritance?

A Generalizations with exceptions are common:

- Most nouns in English aren't marked for CASE, but pronouns are.
- Most verbs in English only distinguish two agreement categories (*3sing* and *non-3sing*), but ***be*** distinguishes more.
- Most prepositions in English are transitive, but ***here*** and ***there*** are intransitive.
- Most nominal words in English are 3rd person, but some (all of them pronouns) are 1st or 2nd person.
- Most proper nouns in English are singular, but some (mountain range names, sports team names) are plural.

Note: defaults are not necessary, and currently implemented grammars do not use default inheritance, though some work is looking into adding it.

Default Inheritance, Technicalities

If a type says $\left[\textsc{arg-st} \; / \left\langle \textsc{np} \right\rangle\right]$, and one of its subtypes says $\left[\textsc{arg-st} \; \left\langle\right\rangle\right]$, then the ARG-ST value of instances of the subtype is $\left\langle\right\rangle$.

If a type says $\left[\textsc{arg-st} \; \left\langle \textsc{np} \right\rangle\right]$, and one of its subtypes says $\left[\textsc{arg-st} \; \left\langle\right\rangle\right]$,

Default Inheritance, Technicalities

If a type says $\left[\text{ARG-ST } /\left\langle \text{NP} \right\rangle\right]$,	and one of its subtypes says $\left[\text{ARG-ST } \left\langle\right\rangle\right]$,	then the ARG-ST value of instances of the subtype is $\left\langle\right\rangle$.
If a type says $\left[\text{ARG-ST } \left\langle \text{NP} \right\rangle\right]$,	and one of its subtypes says $\left[\text{ARG-ST } \left\langle\right\rangle\right]$,	then this subtype can have no instances, since they would have to satisfy contradictory constraints.

In plain English: without the ' / ' the constraint is inviolable. Nothing can be both $\left\langle \text{NP} \right\rangle$ and $\left\langle\right\rangle$, so the subtype is empty.

Default Inheritance, More Technicalities

- If a type says $\left[\textsc{mod}\ /\left\langle \textsc{s}\right\rangle\right]$, and one of its subtypes says $\left[\textsc{mod}\ \left\langle\left[\textsc{spr}\ \left\langle \textsc{np}\right\rangle\right]\right\rangle\right]$, then the $\textsc{mod}$ value of instances of the subtype is what?

Default Inheritance, More Technicalities

- If a type says $\left[\text{MOD } / \left\langle \text{S} \right\rangle\right]$, and one of its subtypes says $\left[\text{MOD } \left\langle \left[\text{SPR } \left\langle \text{NP} \right\rangle\right] \right\rangle\right]$, then the MOD value of instances of the subtype is what?

$$\left[\text{MOD } \left\langle \begin{bmatrix} \text{HEAD} & / \textit{verb} \\ \text{SPR} & \left\langle \text{NP} \right\rangle \\ \text{COMPS} & / \left\langle\right\rangle \end{bmatrix} \right\rangle\right]$$

- That is, default constraints are 'pushed down'

- S abbreviates $\begin{bmatrix} \text{HEAD} & \textit{verb} \\ \text{SPR} & \left\langle\right\rangle \\ \text{COMPS} & \left\langle\right\rangle \end{bmatrix}$, so the subtype only overrides the part it mentions (SPR). The rest of the default survives, still as a default.

Question on Default Inheritance

Q Can a grammar rule override a default constraint on a word?

Question on Default Inheritance

Q Can a grammar rule override a default constraint on a word?

A No. Defaults are all 'cached out' in the lexicon.

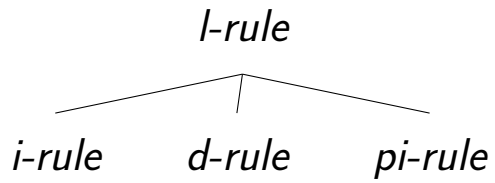
- Words as used to build sentences have only inviolable constraints.
  - i.e. nodes of type *word* in your tree should never be shown with a defeasible constraint (no ' / ' in a tree)

Lexical Rules

Overview

- Recap: the three subtypes of *l-rule*
- The Plural Noun LR, and what it inherits
- Example: licensing ***cats***
- Advice on writing lexical rules
- Constant lexemes
- ARG-ST & ARP; the feature FORM
- Activity: the word structure for ***writers***

Recap: Three Subtypes of *l-rule*



$$\textit{l-rule}: \begin{bmatrix} \textsc{input} & \textit{l-sequence}\ \langle \textsc{x}\ ,\ [\textsc{sem}\ /\ \boxed{2}]\ \rangle \\ \textsc{output} & \textit{l-sequence}\ \langle \textsc{y}\ ,\ [\textsc{sem}\ /\ \boxed{2}]\ \rangle \end{bmatrix}$$

$$\textit{i-rule}: \begin{bmatrix} \textsc{input} & \langle \textsc{x}\ ,\ \begin{bmatrix} \textit{lexeme} \\ \textsc{syn} & \boxed{3} \\ \textsc{arg-st} & \boxed{A} \end{bmatrix} \rangle \\ \textsc{output} & \langle \textsc{y}\ ,\ \begin{bmatrix} \textit{word} \\ \textsc{syn} & \boxed{3} \\ \textsc{arg-st} & \boxed{A} \end{bmatrix} \rangle \end{bmatrix} \qquad \textit{d-rule}: \begin{bmatrix} \textsc{input} & \langle \textsc{x}\ ,\ \begin{bmatrix} \textit{lexeme} \\ \textsc{syn}\ /\ \boxed{3} \end{bmatrix} \rangle \\ \textsc{output} & \langle \textsc{y}\ ,\ \begin{bmatrix} \textit{lexeme} \\ \textsc{syn}\ /\ \boxed{3} \end{bmatrix} \rangle \end{bmatrix}$$

l-rule: the output keeps the input's meaning, by default. *i-rule*: the output is a *word* with the same SYN and ARG-ST, no exceptions. *d-rule*: the output is another *lexeme* with the same SYN, by default.

Plural Noun LR

$$\left[\begin{array}{l} \textit{i-rule} \\ \textsc{input} \quad \langle \boxed{1} \ , \ \textit{cntn-lxm} \rangle \\ \textsc{output} \quad \left\langle \mathrm{F}_{NPL}(\boxed{1}), \left[\begin{array}{l} \textit{word} \\ \textsc{syn} \ \left[\textsc{head} \ \left[\textsc{agr} \ \left[\textsc{num} \ \textit{pl}\right]\right]\right] \end{array}\right] \right\rangle \end{array}\right]$$

Take any count noun lexeme $\boxed{1}$, add the plural ending to its form, and make it a word whose number is plural. That is all the rule itself says; everything else comes from the types.

Practicalities — Applying Lexical Rules

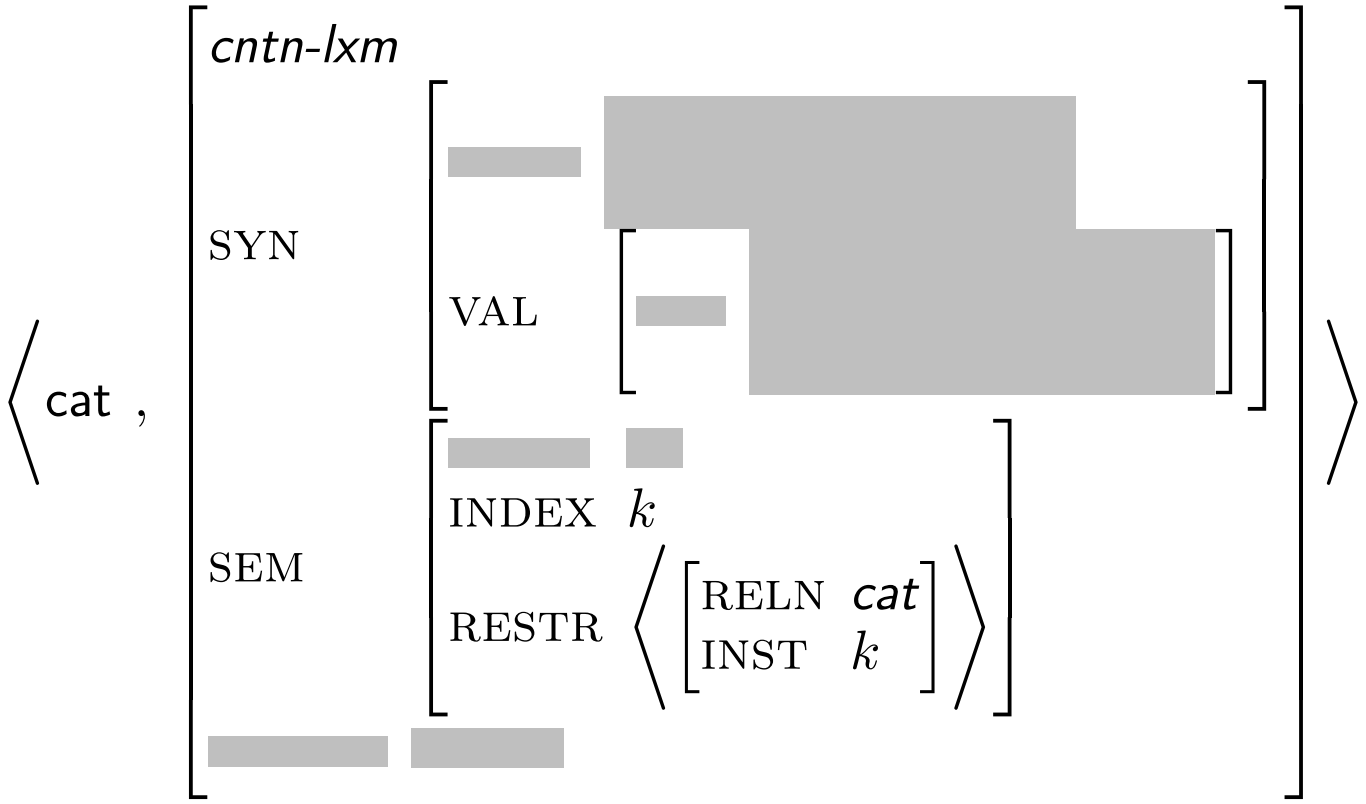
- INPUT is a family of lexical sequences.
- OUTPUT is another family of lexical sequences.
  - ...usually a smaller family
  - ...usually a disjoint one
- The only differences between the families are those stipulated in the rule (or the rule's type).
- Similarities are handled by the constraints on *l-rule* and its subtypes.
- If we've written the LRs correctly, nothing is left underconstrained.

Example: Lexical Entry for *cat*

$$\left\langle \text{cat} \ , \begin{bmatrix} \textit{cntn-lxm} \\ \text{SEM} \begin{bmatrix} \text{INDEX} & k \\ \text{RESTR} & \left\langle \begin{bmatrix} \text{RELN} & \textit{cat} \\ \text{INST} & k \end{bmatrix} \right\rangle \end{bmatrix} \end{bmatrix} \right\rangle$$

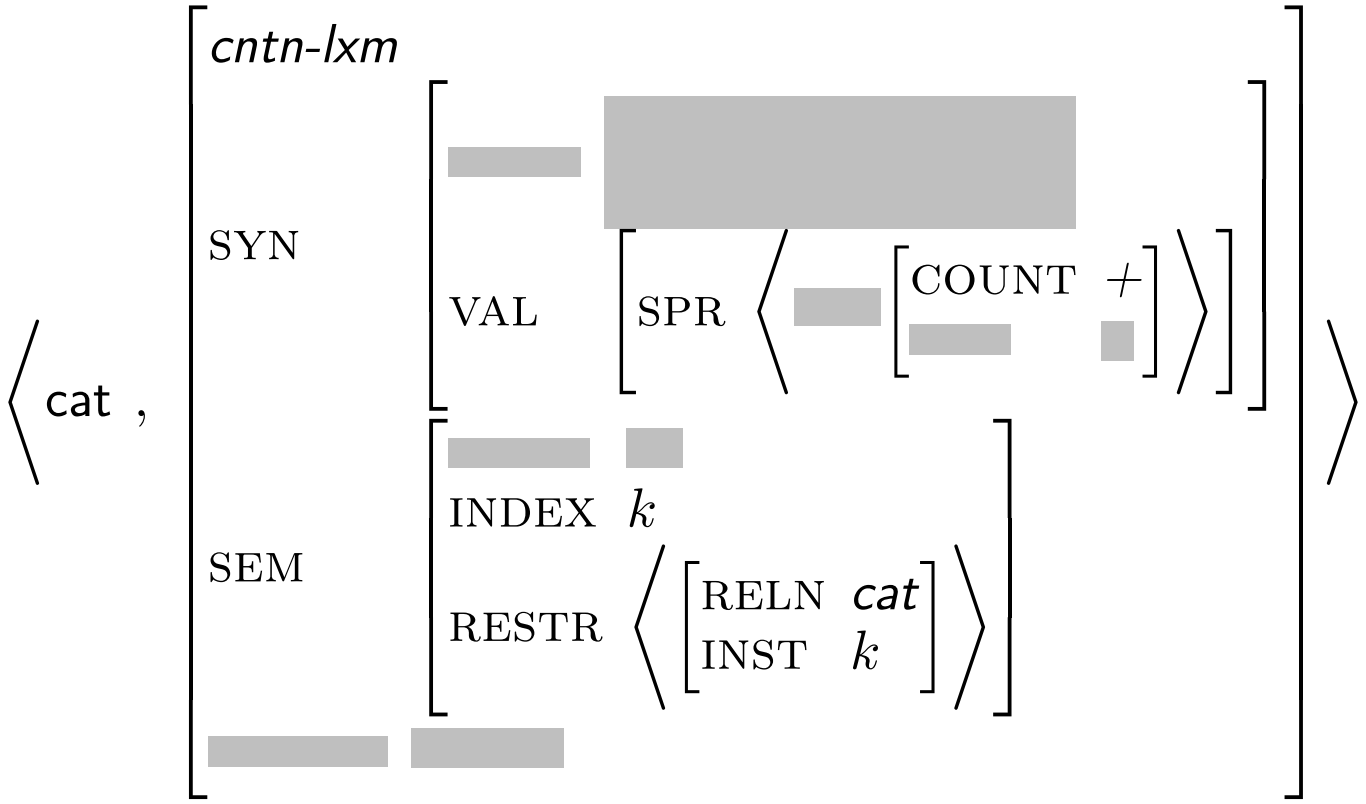
The entry gives only the form, the type and the meaning.

Example: *cat*, with Inheritance



The types fill in the rest. Which type adds what?

Example: *cat*, with Inheritance — *cntn-lxm*



cntn-lxm: the specifier is [COUNT +].

Example: *cat*, with Inheritance — *cn-lxm*

$$\left\langle \text{cat} \ , \begin{bmatrix} \textit{cntn-lxm} & & \\ \text{SYN} & \text{HEAD} & \begin{bmatrix} \textit{noun} \\ \text{AGR} \ \boxed{} \ \begin{bmatrix} \text{PER} & \textit{3rd} \end{bmatrix} \end{bmatrix} \\ & \text{VAL} & \begin{bmatrix} \text{SPR} \left\langle \text{DP} \begin{bmatrix} \text{COUNT} & + \\ \boxed{} & \boxed{} \end{bmatrix} \right\rangle \end{bmatrix} \\ \text{SEM} & & \begin{bmatrix} \text{MODE} & \textit{ref} \\ \text{INDEX} & k \\ \text{RESTR} & \left\langle \begin{bmatrix} \text{RELN} & \textit{cat} \\ \text{INST} & k \end{bmatrix} \right\rangle \end{bmatrix} \\ \text{ARG-ST} & \langle \text{X} \rangle & \end{bmatrix} \right\rangle$$

cn-lxm: *noun*, [PER 3rd], a DP specifier, [MODE ref] and one argument. **Defeasible constraints are inviolable in lexical entries!** The type says [MODE / ref]; nothing in the entry overrides it, so for ***cat*** it is simply [MODE ref].

Example: *cat*, with Inheritance — *infl-lxm*

$$\left\langle \text{cat} \ , \begin{bmatrix} \textit{cntn-lxm} & & \\ \text{SYN} & \text{HEAD} & \begin{bmatrix} \textit{noun} \\ \text{AGR} \ \boxed{7} \ \begin{bmatrix} \text{PER} \ \textit{3rd} \end{bmatrix} \end{bmatrix} \\ & \text{VAL} & \begin{bmatrix} \text{SPR} \ \left\langle \text{DP} \begin{bmatrix} \text{COUNT} & + \\ \text{AGR} & \boxed{7} \end{bmatrix} \right\rangle \end{bmatrix} \\ \text{SEM} & \begin{bmatrix} \text{MODE} & \textit{ref} \\ \text{INDEX} & k \\ \text{RESTR} & \left\langle \begin{bmatrix} \text{RELN} & \textit{cat} \\ \text{INST} & k \end{bmatrix} \right\rangle \end{bmatrix} \\ \text{ARG-ST} & \langle \text{X} \rangle & \end{bmatrix} \right\rangle$$

infl-lxm: the SHAC identifies the noun's AGR with its specifier's, $\boxed{7}$.

Plural Noun LR

$$\left[\begin{array}{l} \textit{i-rule} \\ \textsc{input} \quad \langle \boxed{1} \ , \ \textit{cntn-lxm} \rangle \\ \textsc{output} \quad \left\langle \mathrm{F}_{NPL}(\boxed{1}), \left[\begin{array}{l} \textit{word} \\ \textsc{syn} \ \left[\textsc{head} \ \left[\textsc{agr} \ \left[\textsc{num} \ \textit{pl}\right]\right]\right] \end{array}\right] \right\rangle \end{array}\right]$$

Now feed ***cat*** to the rule.

Licensing *cats*: the Plural Noun LR's Own Contributions

$$\left[\begin{array}{l}\textit{i-rule}\\ \text{INPUT}\quad \left\langle \boxed{1}\text{cat}\ ,\ \left[\begin{array}{l}\textit{cntn-lxm}\\ \text{SYN}\quad \boxed{3}\ \left[\begin{array}{ll}\text{HEAD} & \left[\begin{array}{l}\textit{noun}\\ \text{AGR}\ \boxed{7}\ \left[\text{PER}\ \textit{3rd}\right]\end{array}\right]\\ \text{VAL} & \left[\text{SPR}\ \left\langle \text{DP}\left[\begin{array}{ll}\text{COUNT} & +\\ \text{AGR} & \boxed{7}\end{array}\right]\right\rangle\right]\end{array}\right]\\ \text{SEM}\quad \boxed{2}\ \left[\begin{array}{l}\text{MODE}\ \ \textit{ref}\\ \text{INDEX}\ \ k\\ \text{RESTR}\ \left\langle\left[\begin{array}{ll}\text{RELN} & \textit{cat}\\ \text{INST} & k\end{array}\right]\right\rangle\end{array}\right]\\ \text{ARG-ST}\ \boxed{B}\ \langle \text{X}\rangle \oplus \boxed{C}\ \langle\rangle\end{array}\right]\right\rangle\\ \text{OUTPUT}\quad \left\langle \text{F}_{NPL}\left(\boxed{1}\right)\ ,\ \left[\begin{array}{l}\textit{word}\\ \text{SYN}\quad \boxed{3}\ \left[\begin{array}{ll}\text{HEAD} & \left[\text{AGR}\ \left[\text{NUM}\ \textit{pl}\right]\right]\\ \text{VAL} & \left[\begin{array}{ll}\text{SPR} & \boxed{B}\\ \text{COMPS} & \boxed{C}\end{array}\right]\end{array}\right]\\ \text{SEM}\quad \boxed{2}\\ \text{ARG-ST}\ \boxed{B}\oplus\boxed{C}\end{array}\right]\right\rangle\end{array}\right]$$

In blue: what the rule and its output type add; the rest is copied or inherited.

cats: The (Family of) Lexical Sequence(s)

$$\left\langle \text{cats} \;,\; \left[\begin{array}{ll} \textit{word} & \\ \text{SYN} & \left[\begin{array}{ll} \text{HEAD} & \left[\begin{array}{l} \textit{noun} \\ \text{AGR}\ \boxed{7}\ \textit{3pl} \end{array}\right] \\ \text{VAL} & \left[\begin{array}{ll} \text{SPR} & \boxed{\text{B}} \left\langle \text{DP} \left[\begin{array}{ll} \text{COUNT} & + \\ \text{AGR} & \boxed{7} \end{array}\right] \right\rangle \\ \text{COMPS} & \langle\rangle \end{array}\right] \end{array}\right] \\ \text{SEM} & \left[\begin{array}{ll} \text{MODE} & \textit{ref} \\ \text{INDEX} & k \\ \text{RESTR} & \left\langle \left[\begin{array}{ll} \text{RELN} & \textit{cat} \\ \text{INST} & k \end{array}\right] \right\rangle \end{array}\right] \\ \text{ARG-ST} & \boxed{\text{B}} \end{array}\right] \right\rangle$$

Strictly a family: the DP in SPR is not fully resolved, so this stands for every lexical sequence that fills it in.

Practicalities — Writing Lexical Rules

- Determine the type of the LR.
- Determine the class of possible inputs.
- Determine what should change.
  - If INPUT and OUTPUT values are identified (by default or otherwise) and only OUTPUT value is mentioned, then...
    information is added. e.g. Plural Noun LR

    (Lexical sequences incompatible with that value are not possible inputs)
  - If INPUT and OUTPUT values are identified by default, but different values are given on the INPUT and OUTPUT of the rule, then...
    information is changed. e.g. Agent Nominalization LR
  - If INPUT and OUTPUT values are identified by an inviolable constraint, but different values are given on the INPUT and OUTPUT of the rule, then...
    there is no well-formed input: the rule can never apply.

Constant Lexemes

- What kinds of words are constant lexemes in our grammar?
- Why do we need a rule for these words?
- What would be an alternative analysis?

Constant Lexemes

- What kinds of words are constant lexemes in our grammar?
  Adjectives, adverbs, prepositions, determiners, conjunctions: words that do not inflect in English.

- Why do we need a rule for these words?
  They need to go 'from' *lexeme* to *word*, and only words appear in trees.

- What would be an alternative analysis?
  Put the sub-hierarchy for constant lexemes under *word* instead of under *lexeme*.

Constant Lexeme LR

$$\begin{bmatrix} \textit{i-rule} \\ \textsc{input} & \langle \boxed{1}, \textit{const-lxm} \rangle \\ \textsc{output} & \begin{bmatrix} \textsc{first} & \boxed{1} \end{bmatrix} \end{bmatrix} \quad \text{i.e.} \quad \begin{bmatrix} \textit{i-rule} \\ \textsc{input} & \langle \boxed{1}, \textit{const-lxm} \rangle \\ \textsc{output} & \langle \boxed{1}, [] \rangle \end{bmatrix}$$

The form does not change, and the rule says nothing else. The type *i-rule* does the work: it copies SYN and ARG-ST and makes the output a *word*.

- What keeps this from applying to, say, verb lexemes?
- Why is this an *i-rule*?

Constant Lexeme LR

$$\begin{bmatrix} \textit{i-rule} \\ \text{INPUT} & \langle \boxed{1}\ ,\ \textit{const-lxm} \rangle \\ \text{OUTPUT} & \left[\text{FIRST}\ \boxed{1}\right] \end{bmatrix} \quad \text{i.e.} \quad \begin{bmatrix} \textit{i-rule} \\ \text{INPUT} & \langle \boxed{1}\ ,\ \textit{const-lxm} \rangle \\ \text{OUTPUT} & \langle \boxed{1}\ ,\ [\,] \rangle \end{bmatrix}$$

The form does not change, and the rule says nothing else. The type *i-rule* does the work: it copies SYN and ARG-ST and makes the output a *word*.

- What keeps this from applying to, say, verb lexemes?
  The input must be a *const-lxm*. Verb lexemes are *infl-lxm*s, a different branch of the hierarchy.

- Why is this an *i-rule*?
  It takes a *lexeme* to a *word*, which is what inflectional rules do, even though nothing is inflected.

arg-st & ARP

- Given the ARP, what do we need to specify about the valence properties of words?
- Why isn't the ARP a constraint on the type *lexeme*?

arg-st & ARP

- Given the ARP, what do we need to specify about the valence properties of words?
  Only the length of the SPR list, and the SHAC already does that: nouns and verbs have exactly one specifier. The ARP then fills SPR and COMPS from ARG-ST.

- Why isn't the ARP a constraint on the type *lexeme*?
  Some derivational rules change the argument structure (e.g. Agent Nominalization). If SPR and COMPS were already linked to ARG-ST in the lexeme, those rules would have to change all three.

The Feature form

- Different inflected forms of verbs show up in different syntactic environments. Examples?

- These different forms are syntactically distinguished by the feature FORM, as assigned by lexical rules.

- FORM is also useful in our analyses of coordination and PP selection.

The Feature form

- Different inflected forms of verbs show up in different syntactic environments. Examples?
  I eat; *Eat!*; *I was eating*; *I have eaten*

- These different forms are syntactically distinguished by the feature FORM, as assigned by lexical rules.

- FORM is also useful in our analyses of coordination and PP selection.

What Rules These Out?

(1) **Kim eat pizza.*

(2) **Kim seems to eats pizza.*

(3) **Dana helped Leslie pack and moved.*

(4) **Kim relies for Sandy.*

(5) **Dana walked and Kim.*

What Rules These Out?

(6) **Kim eat pizza.*
eat is either Non-3rd-Singular (does not agree with Kim) or Base Form (not finite, so it cannot head a sentence)

(7) **Kim seems to eats pizza.*
Spoiler: to will be an auxiliary verb that takes a [FORM base] complement (Chapter 12), so eats does not fit

(8) **Dana helped Leslie pack and moved.*
helped selects a [FORM base] complement; the Coordination Rule makes the conjuncts and the mother share FORM, so moved cannot join pack

(9) **Kim relies for Sandy.*
relies requires a PP complement headed by a preposition with [FORM on]

(10) **Dana walked and Kim.*
Used to be fine, since the VAL values match; now the Coordination Rule requires the conjuncts to share FORM, and Kim has none

Review: Lexical Terminology

- **Lexeme**: a type in our grammar; its subtypes state generalizations about combinations of constraints
- **Lexical entry**: what we write in the lexicon; it licenses a family of lexical sequences
- **Lexical sequence**: a form paired with a fully resolved feature structure of type *lexeme* or *word*
- **Family of lexical sequences**: a form paired with an underspecified feature structure, resolvable in many ways
- **Lexical rule**: what we write to derive new lexical sequences from old ones
- **Lexical rule instantiation**: a fully resolved *l-rule*: one rule, one input, one output
- **Word structure**: the bottom of the tree: a form with a fully resolved *word* feature structure over it

Activity: Arriving at a Word Structure

Write the word structure for ***writers*** as it appears in the tree for the sentence

(11) *Those writers need inspiration.*

starting from the lexical entry for ***write***:

$$\left\langle \text{write} \ , \begin{bmatrix} \textit{stv-lxm} & & \\ \text{SEM} & \begin{bmatrix} \text{INDEX} & s & \\ \text{RESTR} & \left\langle \begin{bmatrix} \text{RELN} & \textit{write} \\ \text{SIT} & s \\ \text{WRITER} & i \\ \text{WRITTEN} & j \end{bmatrix} \right\rangle \end{bmatrix} \\ \text{ARG-ST} & \langle \text{X}_i, \ \text{Y}_j \rangle & \end{bmatrix} \right\rangle$$

Activity: Arriving at a Word Structure

Write the word structure for ***writers*** as it appears in the tree for the sentence

(12) *Those writers need inspiration.*

starting from the lexical entry for ***write***:

$$\left\langle \text{write} \ , \begin{bmatrix} \textit{stv-lxm} \\ \text{SEM} & \begin{bmatrix} \text{INDEX} & s \\ \text{RESTR} & \left\langle \begin{bmatrix} \text{RELN} & \textit{write} \\ \text{SIT} & s \\ \text{WRITER} & i \\ \text{WRITTEN} & j \end{bmatrix} \right\rangle \end{bmatrix} \\ \text{ARG-ST} & \langle \text{X}_i, \ \text{Y}_j \rangle \end{bmatrix} \right\rangle$$

Which lexical rules do we need, and in which order? Agent Nominalization LR (***write*** $\rightarrow$ ***writer***, a *d-rule*), then Plural Noun LR (***writer*** $\rightarrow$ ***writers***, an *i-rule*).

Step 1: Agent Nominalization LR

$$\left\langle \text{writer} \ , \begin{bmatrix} \textit{cntn-lxm} & & \\ \text{SYN} & \text{HEAD} & \begin{bmatrix} \textit{noun} \\ \text{AGR} \ \boxed{1} \begin{bmatrix} \text{PER} & \textit{3rd} \end{bmatrix} \end{bmatrix} \\ & \text{VAL} & \begin{bmatrix} \text{SPR} & \left\langle \text{DP} \begin{bmatrix} \text{COUNT} & + \\ \text{AGR} & \boxed{1} \end{bmatrix} \right\rangle \end{bmatrix} \\ \text{SEM} & \begin{bmatrix} \text{MODE} & \textit{ref} \\ \text{INDEX} & i \\ \text{RESTR} & \left\langle \begin{bmatrix} \text{RELN} & \textit{write} \\ \text{SIT} & s \\ \text{WRITER} & i \\ \text{WRITTEN} & j \end{bmatrix} \right\rangle \end{bmatrix} \\ \text{ARG-ST} & \langle \text{DP} \ (, \ \text{PP}_j \begin{bmatrix} \text{FORM} & \textit{of} \end{bmatrix}) \rangle \end{bmatrix} \right\rangle$$

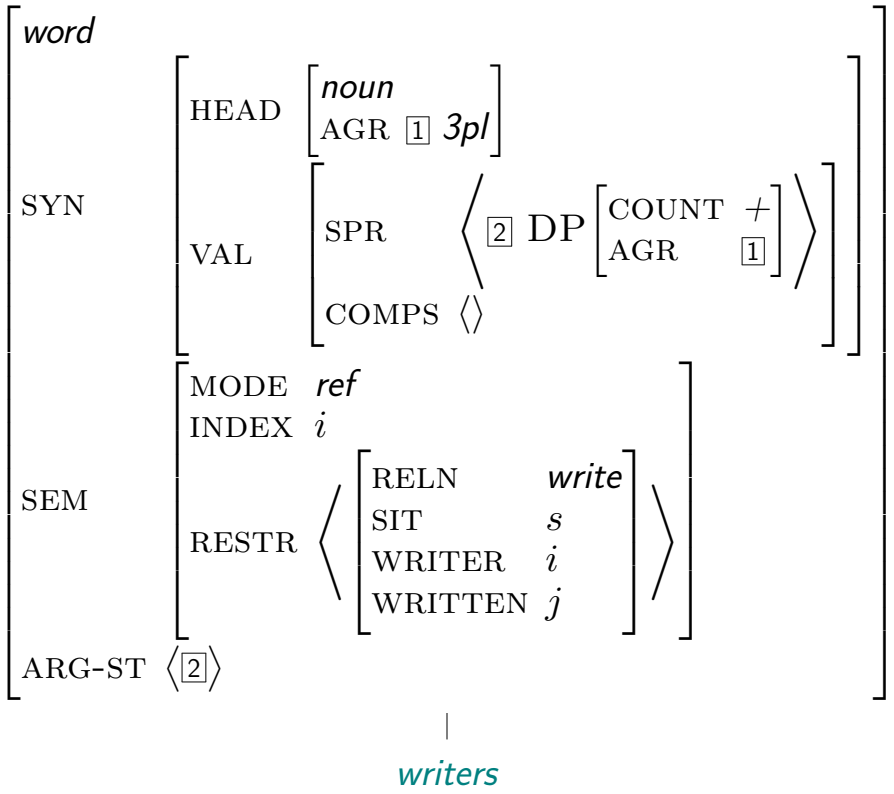
The rule adds *-er*, changes the type to *cntn-lxm* and makes the index the writer, i; the RESTR is unchanged. The noun types add the rest. The verb's object j survives only as an optional *of*-PP.

Step 2: Plural Noun LR

$$\left\langle \text{writers} \ , \begin{bmatrix} \textit{word} \\ \text{SYN} & \begin{bmatrix} \text{HEAD} & \begin{bmatrix} \textit{noun} \\ \text{AGR} \ \boxed{1} \ \textit{3pl} \end{bmatrix} \\ \text{VAL} & \begin{bmatrix} \text{SPR} & \boxed{\text{B}} \left\langle \text{DP} \begin{bmatrix} \text{COUNT} & + \\ \text{AGR} & \boxed{1} \end{bmatrix} \right\rangle \\ \text{COMPS} & \boxed{\text{C}} \ \langle\rangle \end{bmatrix} \end{bmatrix} \\ \text{SEM} & \begin{bmatrix} \text{MODE} & \textit{ref} \\ \text{INDEX} & i \\ \text{RESTR} & \left\langle \begin{bmatrix} \text{RELN} & \textit{write} \\ \text{SIT} & s \\ \text{WRITER} & i \\ \text{WRITTEN} & j \end{bmatrix} \right\rangle \end{bmatrix} \\ \text{ARG-ST} & \boxed{\text{B}} \oplus \boxed{\text{C}} \end{bmatrix} \right\rangle$$

Add *-s* and make it a *word* with [NUM pl]. Now the ARP applies: SPR is the DP and COMPS is empty, since here ***writers*** has no *of*-PP.

Activity: The Word Structure for *writers*



The form *writers* under a feature structure of type *word*, licensed by two lexical rules from the entry for ***write***.

Summary

- A lexical rule says only what changes; its type (*i-rule*, *d-rule*, *pi-rule*) and the input's lexeme type supply the rest, and defaults let a rule override what it needs to
- Licensing a word: lexical entry $\rightarrow$ lexical rule(s) $\rightarrow$ word structure at the bottom of the tree
- Constant lexemes still pass through an *i-rule* to become words
- The ARP links ARG-ST to SPR and COMPS on words; FORM records which inflected form a word is
- Next: the passive, a derivational rule that rearranges ARG-ST

Appendix: The Lexical Rules So Far

Other Lexical Rules (So Far)

- Inflectional (*i-rule*): Singular Noun, 3rd-Singular Verb, Non-3rd-Singular Verb, Past-Tense Verb, Base Form
- Derivational (*d-rule*): Present Participle, Past Participle, Agent Nominalization
- Each one is in the grammar summary in Chapter 9. For reference, on the next slides, read each rule and ask: what is the input, what changes, and where does the rest come from?

Singular Noun Lexical Rule

$$\left[\begin{array}{ll} \textit{i-rule} & \\ \textsc{input} & \left\langle \boxed{1} \text{ , } \textit{cn-lxm} \right\rangle \\ \textsc{output} & \left\langle \boxed{1} \text{ , } \left[\textsc{syn} \left[\textsc{head} \left[\textsc{agr} \left[\textsc{num} \; \textit{sg}\right]\right]\right]\right] \right\rangle \end{array}\right]$$

Any common noun lexeme, count or mass, gives a singular word with the same form. Compare the Plural Noun LR, whose input is only *cntn-lxm*: mass nouns have no plural.

3rd-Singular Verb Lexical Rule

$$\left[\begin{array}{l}\textit{i-rule}\\ \textsc{input} \quad \left\langle \boxed{3}\ ,\ \left[\begin{array}{l}\textit{verb-lxm}\\ \textsc{sem}\ \left[\textsc{restr}\ \boxed{A}\right]\end{array}\right]\right\rangle \\ \textsc{output} \quad \left\langle \mathrm{F}_{3SG}\left(\boxed{3}\right),\ \left[\begin{array}{ll}\textsc{syn} & \left[\textsc{head}\ \left[\begin{array}{ll}\textsc{form} & \textit{fin}\\ \textsc{agr} & \textit{3sing}\end{array}\right]\right]\\ \textsc{sem} & \left[\textsc{restr}\ \boxed{A} \oplus \ldots\right]\\ \textsc{arg-st} & \left\langle\left[\textsc{case}\ \textit{nom}\right],\ \ldots\right\rangle\end{array}\right]\right\rangle\end{array}\right]$$

Add *-s*; the word is finite and 3rd singular, its subject (the first argument) is nominative, and the RESTR gets something extra (the tense).

Non-3rd-Singular Verb Lexical Rule

$$\left[\begin{array}{ll} \textit{i-rule} & \\ \text{INPUT} & \left\langle \boxed{1}\ ,\ \left[\begin{array}{ll} \textit{verb-lxm} & \\ \text{SEM} & \left[\text{RESTR}\ \boxed{A}\right] \end{array}\right] \right\rangle \\ \text{OUTPUT} & \left\langle \boxed{1}\ ,\ \left[\begin{array}{ll} \text{SYN} & \left[\text{HEAD} \left[\begin{array}{ll} \text{FORM} & \textit{fin} \\ \text{AGR} & \textit{non-3sing} \end{array}\right]\right] \\ \text{SEM} & \left[\text{RESTR}\ \boxed{A} \oplus \ldots\right] \\ \text{ARG-ST} & \left\langle \left[\text{CASE}\ \textit{nom}\right], \ldots \right\rangle \end{array}\right] \right\rangle \end{array}\right]$$

The same, but the form stays as it is and the agreement is anything but 3rd singular: *I/you/we/they walk*.

Past-Tense Verb Lexical Rule

$$\left[\begin{array}{l} \textit{i-rule} \\ \textsc{input} \quad \left\langle \boxed{3}\ ,\ \left[\begin{array}{l}\textit{verb-lxm}\\ \textsc{sem}\ \left[\textsc{restr}\ \boxed{A}\right]\end{array}\right]\right\rangle \\ \textsc{output} \quad \left\langle \mathrm{F}_{PAST}\left(\boxed{3}\right),\ \left[\begin{array}{ll}\textsc{syn} & \left[\textsc{head}\ \left[\textsc{form}\ \textit{fin}\right]\right]\\ \textsc{sem} & \left[\textsc{restr}\ \boxed{A} \oplus \ldots\right]\\ \textsc{arg-st} & \left\langle\left[\textsc{case}\ \textit{nom}\right],\ \ldots\right\rangle\end{array}\right]\right\rangle \end{array}\right]$$

Add *-ed*; finite, nominative subject, but no agreement constraint at all: *I/you/she/they walked*.

Base Form Lexical Rule

$$\begin{bmatrix} \textit{i-rule} \\ \textsc{input} & \langle \boxed{1}, \textit{verb-lxm} \rangle \\ \textsc{output} & \langle \boxed{1}, \left[\textsc{syn} \left[\textsc{head} \left[\textsc{form} \ \textit{base}\right]\right]\right] \rangle \end{bmatrix}$$

Same form, [FORM base]: not finite, so no nominative subject and no agreement. This is the verb in *Eat!*, *to eat* and *will eat*.

Present Participle Lexical Rule

$$\left[\begin{array}{l} \textit{d-rule} \\ \textsc{input} \quad \left\langle \boxed{3}\ ,\ \left[\begin{array}{ll} \textit{verb-lxm} & \\ \textsc{sem} & \left[\textsc{restr}\ \boxed{A}\right] \\ \textsc{arg-st} & \boxed{B} \end{array}\right] \right\rangle \\ \textsc{output} \quad \left\langle \mathrm{F}_{PRP}\left(\boxed{3}\right),\ \left[\begin{array}{ll} \textit{part-lxm} & \\ \textsc{syn} & \left[\textsc{head}\ \left[\textsc{form}\ \textit{prp}\right]\right] \\ \textsc{sem} & \left[\textsc{restr}\ \boxed{A} \oplus \ldots\right] \\ \textsc{arg-st} & \boxed{B} \end{array}\right] \right\rangle \end{array}\right]$$

A *d-rule*: ***eat*** gives the lexeme ***eating***, of type *part-lxm*, with the same arguments. *part-lxm* is a *const-lxm*, so the Constant Lexeme LR then makes it a word.

Past Participle Lexical Rule

$$\left[\begin{array}{l} \textit{d-rule} \\ \textsc{input} \quad \left\langle \boxed{3}\ ,\ \left[\begin{array}{ll} \textit{verb-lxm} & \\ \textsc{sem} & \left[\textsc{restr}\ \boxed{A}\right] \\ \textsc{arg-st} & \boxed{B} \end{array}\right] \right\rangle \\ \textsc{output} \quad \left\langle \mathrm{F}_{PSP}\left(\boxed{3}\right),\ \left[\begin{array}{ll} \textit{part-lxm} & \\ \textsc{syn} & \left[\textsc{head}\ \left[\textsc{form}\ \textit{psp}\right]\right] \\ \textsc{sem} & \left[\textsc{restr}\ \boxed{A} \oplus \ldots\right] \\ \textsc{arg-st} & \boxed{B} \end{array}\right] \right\rangle \end{array}\right]$$

The same, with [FORM psp]: ***eat*** gives ***eaten***, as in *I have eaten*.

Agent Nominalization Lexical Rule

$$\left[\begin{array}{l} \textit{d-rule} \\ \textsc{input} \quad \left\langle \boxed{2}\ , \left[\begin{array}{ll} \textit{verb-lxm} & \\ \textsc{sem} & \left[\textsc{index}\ s\right] \\ \textsc{arg-st} & \left\langle \mathrm{X}_i\ ,\ \mathrm{NP}_j \right\rangle \end{array}\right] \right\rangle \\ \textsc{output} \quad \left\langle \mathrm{F}_{\textit{-er}}\left(\boxed{2}\right),\ \left[\begin{array}{ll} \textit{cntn-lxm} & \\ \textsc{sem} & \left[\textsc{index}\ i\right] \\ \textsc{arg-st} & \left\langle \mathrm{Y}\ \left(,\ \mathrm{PP}_j\left[\textsc{form}\ \textit{of}\right]\right)\right\rangle \end{array}\right] \right\rangle \end{array}\right]$$

Add *-er* to a strictly transitive verb lexeme. The result is a count noun lexeme whose index is the verb's subject index i: a ***driver*** is the one who drives. The verb's object j can only appear as an *of*-PP: *a builder of bridges*.

Acknowledgments and References

- Course design and slides borrow heavily from Emily Bender's course: *Linguistics 566: Introduction to Syntax for Computational Linguistics*
  http://courses.washington.edu/ling566

- The slides on lexical types and lexical rules are adapted from Emily Bender's, with her permission; the feature structures follow Sag et al. (2003), © 2003 CSLI Publications

- This lecture complements Sag et al. (2003, Chapter 8)

*

References

Ivan A. Sag, Tom Wasow, and Emily Bender. 2003. *Syntactic Theory: A Formal Introduction*. CSLI Publications, Stanford, second edition.